

**Department of Biomedical Engineering Technology
The Superior University, Lahore**

Computer Programming

Experiment No.4

Control Structures and Relational Operators

Prepared for:

Eisha Nawaz

By:

Name: Mazen Abdulgani

ID: SU91-BBETM-F25-006

Section: Biomedical Engineering

Semester: Second

Total Marks: 10

Obtained Marks:

Signature:

Date:

Experiment No.4**Control Structures and Relational Operators****(Rubrics)****Name: Mazen Abdulgani****ID: SU91-BBETM-F25-006****A. PSYCHOMOTOR**

Serial No.	Criteria	Allocated Marks	Unacceptable 0%	Poor 25%	Fair 50%	Good 75%	Excellent 100%	Total Obtained
1	Follow Procedures	2	0	0.5	1	1.5	2	
2	Accuracy in Output Results	4	0	1	2	3	4	
Sub-Total		6	Sub Total Marks Obtained in Psychomotor (P)					

B. AFFECTIVE

Serial No.	Criteria	Allocated Marks	Unacceptable 0%	Poor 25%	Fair 50%	Good 75%	Excellent 100%	Total Obtained
1	Assigned tasks	2	0	0.5	1	1.5	2	
2	Lab Report	2	0	0.5	1	1.5	2	
Sub-Total		4	Sub Total Marks Obtained in Affective (A)					

Instructor Name: Miss Eisha Nawaz**Total Marks (P+A):** 10**Instructor Signature:** _____**Obtained Marks (P+A):** _____

Control Structures and Relational Operators

1 LAB OBJECTIVES

- To use relational operators and expressions.
- To use logical operators and expressions.
- To study the nature of the control structures.
- To get basic understanding of nested-if conditions.
- To practice how to use the nested-if.

2 INTRODUCTION TO CONTROL STRUCTURES

Control structures provide alternatives to sequential program execution and are used to alter the sequential flow of execution. The two most common control structures are selection and repetition.

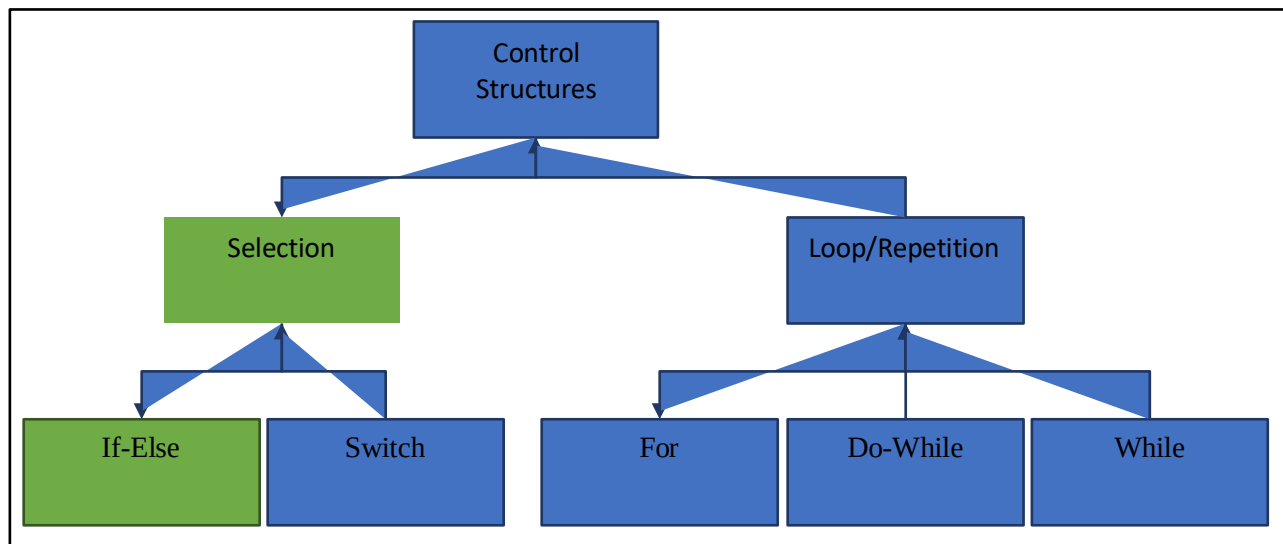


Figure 1 Control Structures

The focus of this lab is to study the If-Else control structure. Remaining control structures will be discussed in next labs.

2.1 SELECTION BASED CONTROL STRUCTURES

In selection, the program executes particular statements depending on some condition(s). The example is shown in Figure 2.

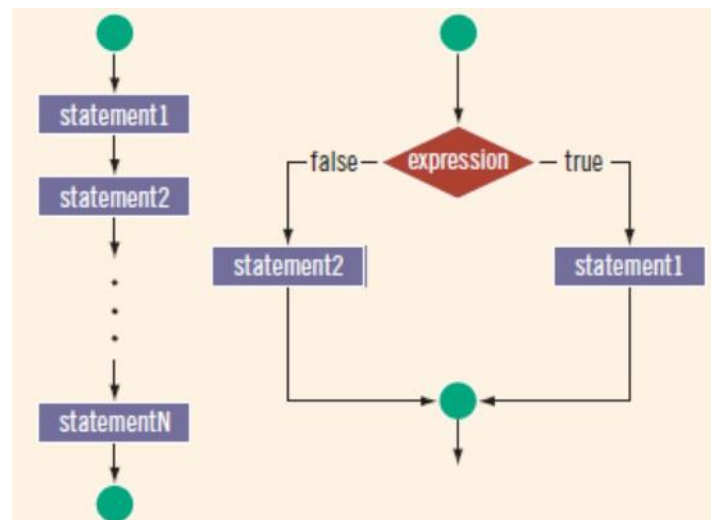


Figure 2 Selection Structure

Before you learn about selection and repetition, you must understand the nature of conditional statements and how to use them.

2.2 ONE WAY SELECTION

It begins with the reserved word `if`, followed by an expression contained within parentheses, followed by a statement. Note that the parentheses around the expression are part of the syntax. The expression is sometimes called a decision maker because it decides whether to execute the statement that follows it. The expression is usually a logical expression. If the value of the expression is true, the statement executes. If the value is false, the statement does not execute and the computer goes on to the next statement in the program. The statement following the expression is sometimes called the action statement.

```
if (expression)
    Statement
```

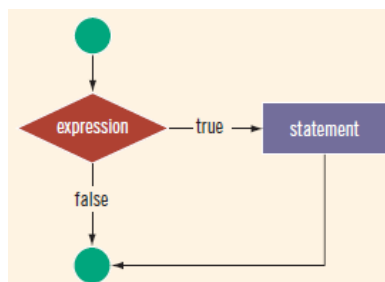


Figure 6

2.2.1 Example

Table 3

	Code	Result
1	//Example 1: One Way Selection.cpp	Please Enter the score: 60
2	#include<iostream>	
3	using namespace std;	

4	int main()	Your entered score is: 60
5	{	Your Grade is: P
6	int score;	
7	char grade = 'P';	
8		
9	cout<<"Please Enter the score: ";	
10	cin>>score;	
11	cout<<"Your entered score is: "<<score<<endl;	
12	if (score>=60){	
13	cout<<"Your Grade is: "<<grade;	
14	cout<<"Your Grade is: "<<grade;	
15	cout<<"Your Grade is: "<<grade;	
	}	
	return 0;	
	}	

2.2.2 Example

	Code	Result
1	//Example 2: Absolute value of an integer	
2	#include <iostream>	
3	using namespace std;	
4	int main()	Please Enter the value of an integer: -4555
5	{	The absolute value of an integer -4555 is: 4555
6	int number, abs;	
7	cout<< "Please enter the value of an integer: ";	
8	cin>> number;	
9	cout<<endl;	
10	abs = number;	
11	if (number < 0)	
12	number = -number;	
13	cout<< "The absolute value of an integer "	
14	<< abs << " is: " << number <<endl;	
15	return 0;	
16	}	
17		

2.3 TWO WAY SELECTION

There are many programming situations in which you must choose between two alternatives. For example, if a part-time employee works overtime, the paycheck is calculated using the overtime payment formula; otherwise, the paycheck is calculated using the regular formula. This is an example of two-way selection. To choose between two alternatives—that is, to implement two-way selections—C++ provides the `if . . . else` statement. Two-way selection uses the following syntax:

```
if (expression)
    Statement 1
else
    Statement 2
```

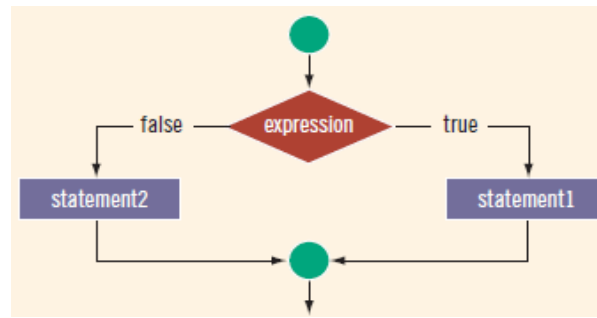


Figure 7

2.3.1 Example

There is a syntax error in the following example.

	Code	Result
1	//Example 3: Absolute value of an integer	Syntax Error
2	if (hours > 40.0) //Line 1	
3	wages = 40.0 * rate + 1.5* rate * (hours-40.0); //line 2	
5	else //Line 3	
6	wages = hours * rate; //Line 4	

The **semicolon at the end of the if statement** (see Line 1) ends the if statement, so the statement in Line 2 separates the else clause from the if statement. That is, else is all by itself. Because there is no stand-alone else statement in C++, this code generates a syntax error. In a one-way selection, the semicolon at the end of an if statement is a logical error, whereas as shown in this example, in a two-way selection, it is a syntax error.

2.3.2 Example

The following program determines an employee's weekly wages. If the hours worked exceed 40, wages include overtime payment.

	Code	Result
1	// Example : Weekly wages	
2	#include <iostream>	
3	using namespace std;	
4	int main ()	
5	{	
6	double wages, rate, hours;	
7	cout<< "Please enter your working hours: ";	
8	cin>> hours;	
9	cout<< "Please enter the rate:Rs ";	
10	cin>> rate;	
11	if (hours > 40.0)	
12	wages = 40.0 * rate + (1.5 * rate * (hours - 40.0));	
13	else	
14	wages = hours * rate;	
15	cout<<endl;	
16	cout<< "Your total calculated wage is: Rs " <<	
17	wages<<endl;	
18	return 0;	

19	}
20	
21	

3 RELATIONAL OPERATORS

C++ includes six relational operators that allow you to state conditions and make comparisons. Each of the relational operators is a binary operator; that is, it requires two operands. Because the result of a comparison is true or false, expressions using these operators evaluate to true or false.

Table 1 Types of Operators in C++

Number of Observations	Operators	Description
1	=	Equal to
2	!=	Not equal to
3	<	Less than
4	<=	Less than or equal to
5	>	Greater than
6	>=	Greater than or equal to

3.1 USING OPERATORS WITH SIMPLE DATA TYPES

You can use the relational operators with all three simple data types. In the following example, the expressions use both integers and real numbers:

Table 2

Expression	Meaning	Value
8 < 15	8 is less than 15	True
6 != 6	6 is not equal to 6	False
2.5 > 5.8	2.5 is greater than 5.8	False
5.9 <= 7.5	5.9 is less than or equal to 7.5	True

3.2 LOGICAL OPERATORS AND EXPRESSIONS

Logical (Boolean) operators enable you to combine logical expressions. C++ has three logical (Boolean) operators.

Operator	Expression
!	Not
&&	And
	Or

Logical operators take only logical values as operands and yield only logical values as results. The operator! is unary, so it has only one operand. The operators && and || are binary operators.

The following panel shows the result of operator && evaluating the expression a&&b:

&& Operator (AND)		
A	B	a&&b
True	False	False
True	True	True

False	False	False
False	True	False

The operator `||` corresponds to the Boolean logical operation OR, which yields true if either of its operands is true, thus being false only when both operands are false. Here are the possible results of `a||b`:

Operator (OR)		
A	B	a&&b
True	False	True
True	True	True
False	False	False
False	True	True

3.3 ORDER OF PRECEDENCE

Complex logical expressions can be difficult to evaluate. Consider the following logical expression:

`11 > 5 || 6 < 15 && 7 >= 8`

This logical expression yields different results, depending on whether `||` or `&&` is evaluated first. If `||` is evaluated first, the expression evaluates to false. If `&&` is evaluated first, the expression evaluates to true. An expression might contain arithmetic, relational, and logical operators, as in the expression:

`5 + 3 <= 9 && 2 > 3`

To work with complex logical expressions, given table shows the Precedence Operators

Operators	Precedence
!, ++, -- (unary operators)	First
*, /, %	Second
+, -	Third
<, <=, >=, >	Fourth
==, !=	Fifth
&&	Sixth
	Seventh

3.3.1 Example 1

Code	Output
<pre>//Program to check if a number is greatest among three numbers using logical && operator int main () { int a,b,c; cout<<" enter three numbers\n"; cin>>a>>b>>c; if (a>b && a>c) cout<<"A is greatest.\n"; else cout<<"A is not greatest\n"; return 0; }</pre>	<pre>Enter three number: 15 12 11 A is greatest</pre>

1 NESTED IF

In the previous lab, you have learned how to use conditions and have learned three different types of conditions (simple if-statement, if-else statement and if-else-if statements, and compound statements). Those statements worked for a single condition; however, there are certain real problems when we need to make the decision based on one two or more conditions. In this case we can use nested-if and will learn in detail about nested-if in this lab. The concept map presented in this lab will guide you when and where to use nested-if.

1.1 FLOWCHART OF NESTED IF

As shown in the given figure, the use of nested if statements help us to check dependency based on the given conditions. For instance, if condition 1 is true then condition 2 will be executed else condition 3 will be executed.

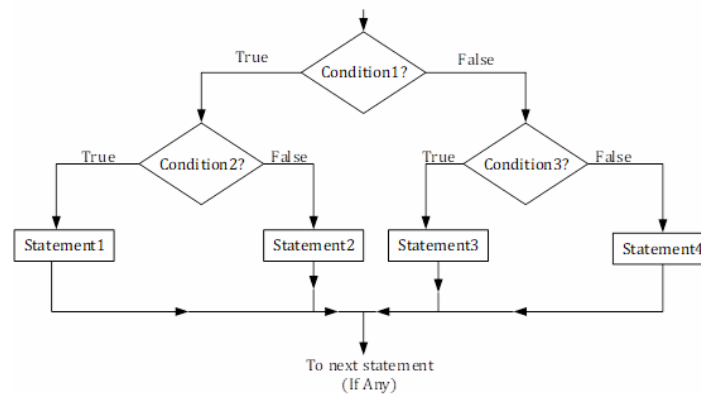


Figure 1 Flowchart of Nested if

1.2 EXAMPLE 1

Code	Output
<pre>// Program to check whether we can go to play cricket or not using nested if #include<iostream> using namespace std; int main () { int temperature; int wind; int friends; cout<<"Enter the temperature: "<<endl; cin>>temperature; cout<<"Enter the Wind/kmh: "<<endl; cin>>wind; cout<<" Enter Number of friends available" <<endl; cin>>friends; if (temperature >= 50) { cout<< "Temperature is fine" <<endl; if (wind <= 40) {</pre>	

```

        cout<<" Wind is fine." <<endl;
        if(friends>=2)
        {
            cout<< "There are enough friends" <<endl;
            cout<<" We can go to play badminton" <<endl;
        }
    }
}
return 0;
}

```

2 IF...ELSE LADDER

The if...else statement executes two different codes depending upon whether the test expression is true or false. Sometimes, a choice has to be made from more than 2 possibilities.

The if...else ladder allows you to check between multiple test expressions and execute different statements.

2.1 SYNTAX

```

if (test expression1) {
// statement(s)
}
else if (test expression2) {
// statement(s)
}
else if (test expression3) {
// statement(s)
}
.
.
else {
// statement(s)
}

```

2.2 EXAMPLE 1

Code	Output
<pre> // Program to relate two integers using =, > or < symbol #include <iostream> Using namespace std; int main () { int number1, number2; cout<<"Enter two integers: "<<endl; cin>>number1>>number2; //checks if the two integers are equal. if (number1 == number2) { cout<<"Result:"<<number1<<"="<<number2<<endl; } //checks if number1 is greater than number2. </pre>	<pre> Enter two integers: 12 23 Result: 12 < 23 </pre>

<pre> else if (number1 > number2) { cout<<"Result:"<<number1<<">"<<number2<<endl; } //checks if both test expressions are false else { cout<<"Result:"<<number1<<"<"<<number2<<endl; } return 0; } </pre>	
--	--

2.3 EXAMPLE 2

Code	Output
<pre> int main() { int a,b,c; cout<<"Enter three number: "<<endl; cin>>a>>b>>c; if(a==b && a==c) cout<<"All are equal.\n"; else if(a>b && a>c) cout<<"a is big.\n"; else if(a==b && a>c) cout<<"a & b are big.\n"; else if(b==c && b>a) cout<<"b & c are big.\n"; else if(a==c && a>b) cout<<"a & c are big.\n"; else if(b>c) cout<<"b is big.\n"; else cout<<"c is big.\n"; return 0; } </pre>	Enter three number: 10 15 12 b is big.

4 LAB TASKS

4.1 TASK 1:

You are a player entering the **Number Quest Arena**. To win, you must pass 3 levels by solving number-based challenges. Each level unlocks the next!

Level 1: Battle of Numbers

The game asks you to enter two integers.

The system compares them:

- Displays the greater number
- Displays the smaller number

```

2  #include <iostream>
3  using namespace std;
4  int main()
5  {
6  cout<<"*THE NUMBER QUEST ARENA*\n";
7  cout<<"LEVEL_1:The Battle of Numbers\n";
8  int NUM1, NUM2;
9  int month;
10 int NUMBER;
11 int score=0;
12 cout<<"Enter First integer:";
13 cin>>NUM1;
14 cout<<"Enter Second integer:";
15 cin>>NUM2;
16 if (NUM1>NUM2){
17 cout<<"The greatest number is:"<<NUM1<<endl;
18 cout<<"The smallest number is:"<<NUM2<<endl;
19 }
20 else if (NUM2>NUM1){
21 cout<<"The greatest number is:"<<NUM2<<endl;
22 cout<<"The smallest number is:"<<NUM1<<endl;
23 }
24 else if (NUM1==NUM2) {
25 cout<<"Both numbers are equal\n";
26 }
27 score+=10;
28
29 cout<<"**\n";

```

```

*THE NUMBER QUEST ARENA*
LEVEL_1:The Battle of Numbers
Enter First integer:51
Enter Second integer:47
The greatest number is:51
The smallest number is:47
**

```

Level 2: Month Decoder

- Enter a number between 1–12.
- The system decodes it into a month name:
 - Example: 6 → JUNE
- If the number is invalid → show "Invalid Month"

```

30 cout<<"LEVEL_2 :MONTH DECODER\n";
31 cout << "Enter a number (1-12): ";
32 cin >> month;
33
34 switch(month) {
35     case 1: cout << "JANUARY\n"; break;
36     case 2: cout << "FEBRUARY\n"; break;
37     case 3: cout << "MARCH\n"; break;
38     case 4: cout << "APRIL\n"; break;
39     case 5: cout << "MAY\n"; break;
40     case 6: cout << "JUNE\n"; break;
41     case 7: cout << "JULY\n"; break;
42     case 8: cout << "AUGUST\n"; break;
43     case 9: cout << "SEPTEMBER\n"; break;
44     case 10: cout << "OCTOBER\n"; break;
45     case 11: cout << "NOVEMBER\n"; break;
46     case 12: cout << "DECEMBER\n"; break;
47     default: cout << "Invalid Month\n";
48 }
49 score+=10;
50
51 cout<<"**\n";

```

```

LEVEL_2 :MONTH DECODER
Enter a number (1-12): 11
NOVEMBER
**

```

Level 3: Divisibility Challenge

- Enter a number.
- Check:
 - If divisible by 3
 - Then also check if divisible by 4
- Otherwise → display "Not divisible by 3"

```
52 cout<<"LEVEL_3 :DIVISIBILITY CHALLENGE\n";
53 int num;
54 cout << "\nEnter a number: ";
55 cin >> num;
56
57 if (num % 3 == 0) {
58     cout << "Divisible by 3\n";
59
60     if (num % 4 == 0) {
61         cout << "Also divisible by 4\n";
62         score += 10;
63     } else {
64         cout << "Not divisible by 4\n";
65     }
66 } else {
67     cout << "Not divisible by 3\n";
68 }
69 score+=10;
70 cout << "\nFinal Score: " << score << endl;
71 return 0;
72 }
```

LEVEL_3 :DIVISIBILITY CHALLENGE

Enter a number: 84
Divisible by 3
Also divisible by 4
Final Score: 30

Bonus Feature (Game Flavor)

- Keep a score counter:
 - +10 points for each correct level input
- Display final score at the end

Home Assignment:

2.4 TASK 1:

Ultimate Logic Arena

1. Show at the start of the output screen.

“Welcome to the Ultimate Logic Arena”

You must pass **5 levels** to become the Logic Champion. Each level tests a different concept of if-else control structures.

Level 1: The Number King

- Enter five numbers.
- Using nested if-else, find and display the largest number.

H_task.cpp

```

1  //MAZEN_LAB-4_HOME-TASK
2  #include <iostream>
3  using namespace std;
4  int main ()
5  {
6      cout<<"*The Ultimate Logic Arena*\n";
7      cout<<"LEVEL 1: The Number King\n";
8      int a,b,c,d,e;
9      int score=0;
10     cout<<"Enter five integers: ";
11     cin>>a>>b>>c>>d>>e;
12     int max = a;
13
14     if (b > max) {
15         max = b;
16     }
17     if (c > max) {
18         max = c;
19     }
20     if (d > max) {
21         max = d;
22     }
23     if (e > max) {
24         max = e;
25     }
26
27     cout << "Largest number is: " << max;
28     score += 10;
29

```

```

*The Ultimate Logic Arena*
LEVEL 1: The Number King
Enter five integers: 24
22
30
28
26
Largest number is: 30
**

```

Level 2: Trade Master

- Enter:
 - Cost Price (CP)
 - Selling Price (SP)
- Determine:
 - If profit → display amount
 - If loss → display amount
 - If equal → display "No Profit No Loss"

```

30  cout<<"\n**\n";
31  cout<<"\nLEVEL 2: Trade Master\n";
32  double CP, SP;
33
34  cout << "Enter Cost Price (CP): ";
35  cin >> CP;
36
37  cout << "Enter Selling Price (SP): ";
38  cin >> SP;
39
40  if (SP > CP) {
41      cout << "Profit = " << SP - CP;
42  }
43  else if (CP > SP) {
44      cout << "Loss = " << CP - SP;
45  }
46  else {
47      cout << "No Profit No Loss";
48  }
49
50  score += 10;
51
52

```

```

LEVEL 2: Trade Master
Enter Cost Price (CP): 128.5
Enter Selling Price (SP): 131.7
Profit = 3.2
**

```

Level 3: Grade Evaluator

- Enter student marks (0–100).
- Assign grade using if–else ladder:

Marks Range	Grade
< 50	F
50–59	C
60–69	B
70–79	B+
80–89	A
90–100	A+

```

53 cout<<"\n**\n";
54 cout<<"\nLEVEL 3: Grade Evaluator\n";
55 int marks;
56 cout << "Enter student marks (0 - 100): ";
57 cin >> marks;
58
59 if (marks < 0 || marks > 100) {
60     cout << "Invalid input. Marks must be between 0 and 100." << endl;
61 }
62 else if (marks < 50) {
63     cout << "Grade: F" << endl;
64 }
65 else if (marks <= 59) {
66     cout << "Grade: C" << endl;
67 }
68 else if (marks <= 69) {
69     cout << "Grade: B" << endl;
70 }
71 else if (marks <= 79) {
72     cout << "Grade: B+" << endl;
73 }
74 else if (marks <= 89) {
75     cout << "Grade: A" << endl;
76 }
77 else { // 90-100
78     cout << "Grade: A+" << endl;
79 }
80 score += 10;
81
82

```

```

LEVEL 3: Grade Evaluator
Enter student marks (0 û 100): 87
Grade: A

```

Level 4: Number Analyzer

- Enter an integer:
 - Check if Even or Odd
 - Check if Divisible by 4
 - Display remainder when divided by 3

```

83 cout<<"\n**\n";
84 cout<<"\nLEVEL 4: Number Analyzer\n";
85 int num;
86 cout << "Enter an integer: ";
87 cin >> num;
88
89 if (num % 2 == 0)
90     cout << num << " is Even" << endl;
91 else
92     cout << num << " is Odd" << endl;
93
94 if (num % 4 == 0)
95     cout << num << " is divisible by 4" << endl;
96 else
97     cout << num << " is not divisible by 4" << endl;
98
99
100 cout << "Remainder when divided by 3: " << num % 3 << endl;
101
102 score += 10;
103

```

```

LEVEL 4: Number Analyzer
Enter an integer: 192
192 is Even
192 is divisible by 4
Remainder when divided by 3: 0

```

**

Level 5: Triple Threat

- Enter three numbers
- Find:
 - Largest number
 - Smallest number

```
103
104 cout<<"\n**\n";
105 cout<<"\nLEVEL 5: Triple Threat\n";
106 cout << "\nLevel 5: Enter three numbers: ";
107 cin >> a >> b >> c;
108
109 int largest = a;
110 if (b > largest) largest = b;
111 if (c > largest) largest = c;
112
113 int smallest = a;
114 if (b < smallest) smallest = b;
115 if (c < smallest) smallest = c;
116
117 cout << "Largest number: " << largest << endl;
118 cout << "Smallest number: " << smallest << endl;
119
120 score += 10;
121
122 cout << "\nFinal Score: " << score << endl;
123
124 if (score >= 50)
125     cout << "Title: Logic Champion" << endl;
126 else if (score >= 30)
127     cout << "Title: Skilled Player" << endl;
128 else
129     cout << "Title: Try Again" << endl;
130
131
132 return 0;
133 }
```

LEVEL 5: Triple Threat

```
Level 5: Enter three numbers: 40
38
42
Largest number: 42
Smallest number: 38

Final Score: 50
Title: Logic Champion
-----
```

★ Bonus Game Feature

- Score system:
 - +10 points per level
- Final title:
 - 50 → Logic Champion
 - 30–40 → Skilled Player
 - <30 → Try Again